INT396 · UNSUPERVISED LEARNING

UNIT 4

Dimensionality Reduction and Representation Learning

Too many features makes distance meaningless. These are the techniques for finding the few directions that actually carry the information.

Curse of Dimensionality • PCA - Geometric Intuition • Explained Variance • Manifold Learning • t-SNE • UMAP • Autoencoders

School of Computer Science & Engineering

Session 2026-27 · L:2 T:0 P:2 · Credits: 3

UNIT IV ROADMAP

What This Unit Covers

UNIT 4

From a linear projection you can compute by hand to a neural network that learns its own features.

- Why reduce dimensions at all
  - The curse of dimensionality, measured
  - Noise, compute cost and the impossibility of plotting
- Linear reduction
  - PCA and its geometric intuition
  - Explained variance and choosing how many components
- Non-linear reduction
  - Manifold learning - the idea of an embedded surface
  - t-SNE and UMAP, conceptually
  - What their plots do and do not mean
- Representation learning
  - Autoencoder intuition: compress, then reconstruct
  - Why the bottleneck is the point

Why
the curse of dimensionality

Linear
PCA

Non-linear
manifolds, t-SNE, UMAP

Learned
autoencoders

01

SECTION

Why Reduce Dimensions?

IN THIS SECTION

- The curse of dimensionality
- Why distance stops discriminating
- Noise, compute and storage
- The fact that you cannot plot 60 dimensions

Four Reasons to Reduce Dimensions

Adding a feature is not free.

REASON 1

Distance stops working

- As features grow, all pairwise distances converge
- Nearest and furthest become nearly equal
- Every distance-based method loses its grip

REASON 2

Irrelevant features add noise

- Each one contributes to every distance
- An uninformative feature does not merely fail to help
- It actively dilutes the signal

REASON 3

Compute and storage

- Distance matrices are $O(n^2 d)$
- Fewer features means faster everything
- Often the difference between feasible and not

REASON 4

You cannot see it

- Nobody can picture 60 dimensions
- Two or three can be plotted
- Seeing structure is how you form a hypothesis

PREDICT You have 50 samples and 300 features. What do you expect clustering to produce?

The Curse of Dimensionality

COMMON PITFALL · Why high-dimensional distance becomes meaningless

The phenomenon as the number of features grows, the distance to the nearest point and the distance to the furthest point become almost equal.

Why every extra dimension adds another squared difference to the sum. Across many dimensions those differences average out, so all pairs accumulate a similar total.

The consequence 'nearest neighbour' stops being meaningful. There are no dense regions, because density requires contrast.

What it breaks k-NN, k-Means, DBSCAN, silhouette - everything built on distance.

The symptoms in the lab clusters that look arbitrary, results that change on a re-run, silhouette scores hovering near zero.

The responses select fewer, better features; reduce dimensionality; or use cosine similarity, which holds up better on sparse data.

02

SECTION

Principal Component Analysis

IN THIS SECTION

- The geometric intuition - find the long axis of the cloud
- Components are orthogonal and ordered by variance
- Explained variance ratio and the scree plot
- What PCA costs you

PCA - The Four Steps

Find the directions the data actually varies in, then keep only those.

STEP 1-2 · CENTRE THE DATA, THEN COMPUTE THE COVARIANCE

$$X_c = X - \mu \qquad \Sigma = (1/(n-1)) \cdot X_c^T X_c$$

Centring is essential: PCA finds variance ABOUT THE MEAN, so skipping this gives the wrong axes.

STEP 3 · EIGEN-DECOMPOSITION - THE EIGENVECTORS ARE THE NEW AXES

$$\Sigma v_i = \lambda_i v_i \quad \text{with} \quad \lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d$$

Each eigenvector v_i is a direction; its eigenvalue λ_i is the variance captured along it. Sorting by λ orders the components.

STEP 4 · PROJECT ONTO THE TOP K EIGENVECTORS

$$X_pca = X_c \cdot W_k \quad \text{where} \quad W_k = [\, v_1 \ldots v_k \,]$$

Each row of X_pca is the original point expressed in k numbers instead of d.

EXPLAINED VARIANCE RATIO

$$EVR_i = \lambda_i / \Sigma_j \lambda_j$$

The fraction of total variance captured by component i. The ratios sum to 1 across all components.

THE GEOMETRIC INTUITION

PC1 the direction of maximum variance - the long axis of the cloud

PC2 the direction of most remaining variance, perpendicular to PC1

Orthogonal components are at right angles, so they are uncorrelated

Ordered $\lambda_1 \geq \lambda_2 \geq \ldots$ so early components carry the most

SCALE FIRST PCA maximises variance, and variance depends on units - otherwise the largest-unit feature becomes PC1

Choosing How Many Components to Keep

THE PROBLEM

- A dataset with 13 features
- Eigenvalues give explained variance ratios:
- 0.362, 0.192, 0.111, 0.071, 0.066, 0.049, 0.042, 0.027, ...
- Task: how many components reach 90% of the variance?

WHY IT MATTERS

- Two common rules: a fixed threshold (90% or 95%), or the 'elbow' of the scree plot
- Components are always sorted, so cumulative variance only rises
- Keeping fewer components often IMPROVES clustering by removing noise dimensions

STEP-BY-STEP SOLUTION

Cumulative after 1 $0.362 \rightarrow 36.2\%$

Cumulative after 2 $0.362 + 0.192 = 0.554 \rightarrow 55.4\%$

Cumulative after 3 $0.554 + 0.111 = 0.665 \rightarrow 66.5\%$

Cumulative after 4 $0.665 + 0.071 = 0.736 \rightarrow 73.6\%$

Cumulative after 5 $0.736 + 0.066 = 0.802 \rightarrow 80.2\%$

Cumulative after 6 $0.802 + 0.049 = 0.851 \rightarrow 85.1\%$

Cumulative after 7 $0.851 + 0.042 = 0.893 \rightarrow 89.3\%$

Cumulative after 8 $0.893 + 0.027 = 0.920 \rightarrow 92.0\%$ ✓ first to pass 90%

ANSWER **8 of 13 components carry 90% of the variance - a 38% reduction.**

What PCA Gives You, and What It Costs

A compression that is cheap in accuracy and expensive in explanation.

What you gain

Fewer features often a large reduction with very little information lost.

Uncorrelated components the new axes are orthogonal, which removes multicollinearity.

Noise reduction low-variance directions are usually mostly noise, and dropping them can improve clustering.

Visualisation two components give an honest 2-D shadow of high-dimensional data.

Speed every downstream computation is cheaper.

What you lose

Interpretability PC1 is a MIXTURE of original features, not one of them. You can no longer say 'income drove this'.

Linear only PCA can only rotate and project. Structure that lies on a curved surface is not captured.

Variance is not importance a low-variance feature can still be the informative one.

Requires scaling otherwise the largest-unit feature becomes PC1 automatically.

Regulated settings if you must explain a decision, an un-nameable component may be unacceptable.

03

SECTION

Manifold Learning, t-SNE and UMAP

IN THIS SECTION

- The manifold hypothesis
- Why linear projection is sometimes not enough
- t-SNE - preserving local neighbourhoods
- UMAP - and what these plots do NOT mean

The Manifold Hypothesis

KEY CONCEPT · **High-dimensional data often lies on a low-dimensional surface**

The idea real data rarely fills its high-dimensional space. It tends to lie on or near a much lower-dimensional surface embedded within it.

The standard picture a sheet of paper rolled into a spiral sits in three dimensions, but it is intrinsically two-dimensional. Unroll it and you lose nothing.

Why PCA struggles PCA can only rotate and flatten. It cannot unroll. Projecting a rolled sheet linearly overlaps points that are far apart along the surface.

What manifold learning does it tries to recover the surface itself, preserving distances measured ALONG it rather than through the ambient space.

Why it matters for images a 64-pixel digit lives in 64 dimensions, but the set of all valid digit images is a far smaller surface within that space.

The methods t-SNE and UMAP are the two you will meet - both non-linear, both primarily for visualisation.

PCA vs t-SNE vs UMAP

PCA is a transform you can put in a pipeline. t-SNE is a picture.

	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Preserves	Global variance	Local neighbourhoods	Local, plus some global
Speed	Fast	Slow	Medium
Transform new points?	Yes	No	Yes
Deterministic?	Yes	No - depends on seed	No - depends on seed
Main parameter	n_components	perplexity	n_neighbors, min_dist
Use it for	Preprocessing, compression, pipelines	Visual exploration only	Visualisation, sometimes preprocessing

Three Things a t-SNE Plot Does NOT Tell You

COMMON PITFALL · These plots appear in papers as evidence. They are not

1 - Cluster SIZE means nothing t-SNE expands dense regions and contracts sparse ones. A large blob is not a large cluster.

2 - DISTANCE between clusters means nothing two clusters far apart on the plot are not necessarily more different than two that are close. Global geometry is not preserved.

3 - Apparent clusters may not be real run t-SNE on pure random noise and you will still see distinct clumps.

Perplexity changes the picture low values fragment the data into many small groups; high values merge them. There is no correct value, which is itself a warning.

What to do treat a t-SNE plot as a HYPOTHESIS, never as evidence. Corroborate it with a clustering metric computed in the ORIGINAL space.

UMAP is faster, preserves somewhat more global structure and can transform new points - but carries the same three warnings.

DISCUSS A colleague shows four clear clumps in a t-SNE plot and says 'we found four segments'. What two questions do you ask?

04

SECTION

Autoencoders

IN THIS SECTION

- A neural network that learns to copy its input
- Why the bottleneck is the whole point
- Encoder, latent code, decoder
- How this differs from PCA

How an Autoencoder Works

Trained to reproduce its own input - which sounds pointless until you look at the middle.

01 INPUT

- The original vector
- Say 784 pixels

02 ENCODER

- Layers that shrink the representation
- 784 → 128 → 32

03 BOTTLENECK

- The latent code - say 16 numbers
- Everything must pass through here
- This IS the compressed representation

04 DECODER

- Layers that expand it back
- 16 → 32 → 128 → 784

05 OUTPUT

- A reconstruction of the input
- Loss = how different it is from the original

ASK YOURSELF If the network just has to copy its input, why does it learn anything useful?

Why the Bottleneck Makes It Work

KEY CONCEPT · The constraint is what forces the learning

Without a bottleneck the network could learn the identity function - copy each input straight through - and learn nothing at all.

With a bottleneck the data must be squeezed through far fewer numbers than it started with, so the network cannot simply pass it along.

What it is forced to do discover which aspects of the input matter and encode those, discarding the rest.

The latent code those few numbers are a learned representation - found from the data, with no labels anywhere.

This is unsupervised the 'label' is the input itself, so no annotation is needed. That idea generalises far beyond autoencoders.

Uses compression, denoising, anomaly detection - a point that reconstructs badly is unlike the training data.

How it differs from PCA

- PCA is linear; an autoencoder can learn curved structure
- A linear autoencoder with one layer recovers the PCA subspace
- PCA is deterministic and has a closed-form solution; an autoencoder is trained and depends on initialisation

CHECK YOUR UNDERSTANDING

Quick Check - Unit IV

UNIT 4

QUICK CHECK

You apply PCA to data measured in centimetres, kilograms and rupees WITHOUT standardising first. What is the most likely outcome?

A PCA fails and returns an error

B PC1 is dominated by whichever feature has the largest numerical range

C The components come out in a random order

D It works correctly - PCA is scale-invariant

ANSWER: B PCA maximises variance, and variance depends on the units a feature happens to be recorded in. Rupees will have a far larger numerical variance than kilograms, so PC1 becomes essentially the rupee axis. Standardise first so every feature contributes on equal terms.

Activity: Reduce, Then Justify

GROUP WORK · 12 minutes

In groups of 3-4, plan a dimensionality reduction for one dataset. You are choosing and defending a method, not running code.

1

Pick a dataset

- Student records with 40 fields
- A robot logging 200 sensor channels
- Product reviews as 5,000-term word counts
- Or propose your own

2

Decide

- Do you need reduction at all? Say why
- Linear or non-linear? Justify from the data, not the fashion
- How many components, and by which rule?

3

Anticipate

- What will you lose that you currently rely on?
- Could you still explain a decision afterwards?
- Does anything need scaling first?

4

Argue the other side

- Name the method you rejected and what would go wrong with it

DELIVERABLE One speaker per group: 60 seconds on your method, your component count, and the one thing you gave up.

UNIT IV RECAP

Unit IV - The Six Ideas Worth Remembering

UNIT 4

1. **High dimensions make distance meaningless**
   Nearest and furthest converge, so every distance-based method loses its grip. Adding an irrelevant feature actively dilutes the signal.

2. **PCA finds the directions of maximum variance**
   Centre the data, take the covariance matrix, eigen-decompose it, and project onto the top k eigenvectors. Components are orthogonal and ordered.

3. **Explained variance ratio tells you how many to keep**
   Cumulative EVR against a threshold, or the elbow of the scree plot. Fewer components often clusters better by removing noise.

4. **PCA costs you interpretability**
   A principal component is a mixture of original features, so you can no longer name what drove a decision.

5. **The manifold hypothesis explains why linear is not enough**
   Data often lies on a curved low-dimensional surface. PCA can rotate and flatten but it cannot unroll.

6. **An autoencoder learns a representation through a bottleneck**
   The constraint forces it to keep what matters. The label is the input itself, so nothing needs annotating.

NEXT UP Unit V - Association Rule Mining and Anomaly Detection: support, confidence, lift and conviction; Apriori and FP-Growth; then Isolation Forest and Local Outlier Factor.